

Technical Project Report: Ethical Governance System (EaaS) for Course Enrollment

1. Project Vision and Architectural Decisions

To implement this university enrollment system, I designed and developed a Service-Oriented Architecture (SOA) based on isolated microservices containerized via Docker. I decided to structure the application into four distinct modules to ensure maximum Separation of Concerns and to allow each component to evolve independently, without tight technological constraints other than compliance with the interface contract (JSON payload).

In the Docker Compose orchestration file, I implemented dedicated healthcheck mechanisms. This choice was necessary to guarantee deployment resilience, blocking the initialization of logical modules until the semantic persistence layer is fully active and responsive.

2. The Orchestrating Client and Functional Decoupling

In designing the Client, I intended for this component to act not merely as a terminal, but to assume the crucial role of **Application Flow Orchestrator**. The Client is the entity governing the logical sequence: it queries the data service, normalizes the context, and requests ethical validation.

I structured the Client's code (specifically the `runScenario` function) so that the orchestration logic is completely isolated and independent. This architectural decision guarantees total decoupling: the core functions of the flow are self-contained and abstracted from the user interface. In an industrial evolution of the system, I can easily separate this block from a GUI (Graphical User Interface) or a web front-end; the interface will simply pass the input parameters (Student ID and Course ID) directly to the orchestrator functions, which will remain unchanged and protected within their logical perimeter.

For the Client's development, I chose not to overburden the architecture with heavy frameworks like Spring Boot. I implemented the logic leveraging the native **Java HTTP Client API** (introduced in Java 11), combined with the *Jackson* library for JSON node manipulation. This choice allowed me to write extremely lean and high-performing code, maintaining full control over timeout management and retry policies with exponential backoff.

3. DaaS (Data as a Service) and Semantic Abstraction

In order to manage the university domain information, I configured a graph database based on an **RDF Triplestore** (Apache Jena Fuseki). To prevent the Client from needing to know SPARQL syntax or the details of semantic graph modeling (e.g., the `foaf:Person` or `ex:gpa` predicates), I implemented the **DaaS** microservice.

The DaaS acts as a layer of **semantic abstraction**: it takes on the responsibility of translating REST requests from the orchestrator into complex graph queries, returning a clean and interoperable JSON payload to the Client using the *Apache Jena* library.

4. EaaS (Ethics as a Service): Evolution of Decision Models

The EaaS represents the *Bounded Service Block* dedicated to ethical governance. In the current version, I implemented a **deterministic rule-based decision system**, where an agile engine (Apache Commons JEXL) evaluates "if-then" logical expressions loaded from an external JSON file, guaranteeing the application of the **Zero-Trust Principle** (the EaaS recalculates risk from scratch without trusting external metadata) and traceability through an immutable **Audit Trail**.

However, to overcome the inherent rigidity of fixed rules, I theorized an architectural expansion of the EaaS toward a **Hybrid Decision Agent**, capable of combining and layering four different algorithmic models:

1. **Rule-Based Model**: It remains the foundational and mandatory layer (Guardrail). It verifies absolute ethical and legal constraints (e.g., non-negotiable accessibility), guaranteeing immediate responses such as REJECT or PROCEED.
2. **Model-Based / Predictive Model**: I hypothesized the integration of a Machine Learning component capable of analyzing the University's historical academic data to *predict* the probability of a student's success or dropout if they were to enroll in a high-workload course.
3. **Goal-Based Model**: The system evolves by setting an active goal: *maximize inclusion and degree completion*. The assignment is evaluated not just as a snapshot, but based on how much closer it brings the student to the goal of graduation.
4. **Utility-Based Model**: I planned the introduction of a utility function $U(s, c)$ that assigns a mathematical score to the quality of the enrollment. If for a student with a low GPA a 12-CFU course represents a high risk (low utility), the utility-based system detects if that specific course is the last missing exam before the thesis, pushing the organizational utility to the maximum and overriding the automatic block.

5. EaaS as an Autonomous System: Integrating the MAPE-K Loop

To elevate the EaaS architecture from a simple static executor to a resilient and self-improving system, I theorized its structure as an **Autonomous System guided by the MAPE-K loop** (*Monitor-Analyze-Plan-Execute over Knowledge*). This choice allows the EaaS to monitor the effectiveness of its own ethical decisions and dynamically adapt its behavior at runtime.

I mapped the loop components within the application domain as follows:

- **Monitor**: The system does not just receive a single JSON payload, but constantly monitors the request flow and queries the **Audit Trail** service. It collects aggregated data on how many enrollments are approved (PROCEED), how many are escalated (ESCALATE), and the long-term academic outcome of the students obtained through the

DaaS.

- **Analyze:** In this phase, the EaaS analyzes historical Audit Trail data to identify anomalies or the emergence of "algorithmic brick walls." For example, if the analysis detects that the deterministic rule model is generating an abnormally high REJECT rate for a specific category of students (e.g., working students), the system identifies a potential bias or excessive rigidity in the current policy.
- **Plan:** Once a criticality is detected, the planning module decides which strategy to adopt to optimize ethical governance. I envisioned that in this phase the system dynamically chooses which of the four decision models (rules, models, goals, utility) is most appropriate for the specific context, planning a reconfiguration of the utility function weights or the activation of the RAG module for a deeper semantic evaluation.
- **Execute:** The EaaS performs the reconfiguration at runtime entirely autonomously. It modifies the evaluation order of the decision modules or updates the parameters of the JEXL engine, applying the new strategy without requiring a microservice restart or interrupting the Orchestrating Client.
- **Knowledge:** This is the shared knowledge base fueling the entire loop. It stops being a mere static archive and becomes the core of learning. It contains the RDF datasets, the JSON policies file, and, most importantly, **the entire history of the Audit Trail enriched with human decision logs.**

The Role of the Audit Trail and Human Feedback in Self-Adaptation

The most crucial aspect of this autonomous loop lies in enhancing the *Human-in-the-Loop* through the historicization of the Audit Trail. When the system emits an ESCALATE action, the decision is delegated to a human academic tutor.

I designed the architecture so that the tutor's corrective intervention is not isolated, but is recorded in the Audit Trail as a regulatory feedback event. The MAPE-K loop retroactively analyzes these logs: if the Audit Trail shows that human tutors systematically override a certain algorithmic REJECT to approve the enrollment of working students with profiles similar to Charlie's, the system "understands" the discrepancy.

Through this mechanism, the EaaS self-improves its metrics, autonomously realigning the parameters of the Utility function or suggesting an update to the JEXL rules. In this way, the Audit Trail evolves from an inspectable control tool into a true source of continuous learning, allowing the system to absorb human experience to mitigate its own intrinsic limitations.

6. Technical Debt and Current Limitations

In the current development, I am aware of having left two main elements of controlled technical debt:

- **Synchronous Communication:** The microservices communicate via synchronous HTTP REST calls. In a high-traffic scenario, this temporal coupling could generate bottlenecks. I plan to optimize it by introducing an *Event-Driven* architecture mediated by a Message Broker (e.g., RabbitMQ or Apache Kafka).
- **Implicit Contract Validation:** Data transfer relies on structural trust in the JSON payload. To deploy the system into production, I consider it necessary to implement strict validation schemas (JSON Schema) at the ingress of each *Bounded Context*.